⚡ Scaling, Performance & Reliability: Complete Interview Mastery Guide

Category 8 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic: Simple Definition → Why it exists → Internals → Key concepts → Short interview answer → Deep dive → Real-world example → Interview Q&A → Gotchas → Connections.
>
> ⚠ = Frequently misunderstood or heavily tested.
>
> Category 8 is where your **SRE credentials** are tested. Most engineers know how to write a Jenkinsfile. Senior/staff engineers know how Jenkins performs at scale, how to tune it when it's struggling, how to recover it when it's down, and how to observe it before it breaks. If you're interviewing for a platform team that owns CI/CD infrastructure for 500+ engineers, the interviewer will probe deeply here. The two highest-value topics: JVM tuning (8.1) and monitoring (8.5) — they reveal whether you've ever actually operated Jenkins in production under load.

📑 TABLE OF CONTENTS

1. Topic 8.1 — Jenkins Performance Tuning
2. Topic 8.2 — Build Time Optimization
3. Topic 8.3 — Ephemeral vs Persistent Agents ⚠
4. Topic 8.4 — Jenkins Backup and Recovery
5. Topic 8.5 — Monitoring Jenkins
6. Category 8 Summary & Self-Quiz

Topic 8.1 — Jenkins Performance Tuning

🔴 Advanced | Making Jenkins Fast Under Load

📌 What It Is — In Simple Terms

Jenkins is a Java application running on the JVM. Its performance under load — many concurrent builds, many jobs, many plugins — depends critically on JVM heap settings, garbage collector choice, thread pool configuration, and filesystem behavior. A poorly tuned Jenkins Controller becomes the bottleneck for an entire engineering organization: builds queue instead of starting, the UI becomes unresponsive, and builds fail with OOM errors or timeout exceptions. Performance tuning is the discipline of understanding where the bottleneck is and fixing it systematically.

🔍 Jenkins Performance Failure Modes

```
Symptom                      → Root cause              → Tuning lever
──────────────────────────────────────────────────────────────────────────────
UI unresponsive (30-60s lag) → Heap exhaustion/GC pause → Increase heap,
tune GC
Builds queue, don't start    → Too few executors OR      → Increase agent
capacity
                               Controller executor=0     → Check queue
drain rate
OutOfMemoryError             → Heap too small for state  → Increase -Xmx
"GC overhead limit exceeded" → Full GC consuming >98%    → Tune GC,
increase heap
                               CPU time                  → Check for
memory leak
Controller CPU 100%          → Many concurrent HTTP req  → Tune thread
pool
                               OR runaway Groovy CPS     → Check for
infinite loops
Disk I/O saturation          → MAX_SURVIVABILITY mode    → Switch to
PERF_OPTIMIZED
                               OR too many builds         → Build
discarder cleanup
Slow build scheduling        → Large queue computation   → Reduce labels
complexity
                               OR many pending jobs       → Scale agent
capacity
Plugin load time             → Too many plugins          → Audit and
remove unused
Slow startup (>5 min)        → Large JENKINS_HOME        → Archive/clean
old data
```

⚙️ JVM Heap Settings

```
# ── HEAP SIZING GUIDELINES ──────────────────────────────────────────
# Rule of thumb: Start with 4GB, monitor, increase as needed
# Hard minimum for any non-trivial Jenkins: 2GB
# Small org (<100 jobs, <20 concurrent builds): 4GB
# Medium org (100-500 jobs, <100 concurrent builds): 8-16GB
# Large org (500+ jobs, 100+ concurrent builds): 16-32GB
# Very large (1000+ jobs, 200+ concurrent builds): 32GB+

# JENKINS_JAVA_OPTS or JAVA_OPTS environment variable:
export JAVA_OPTS="\
  -Xms4g \
  -Xmx16g \
  -XX:+UseG1GC \
  -XX:G1HeapRegionSize=32m \
```

```
    -XX:+ParallelRefProcEnabled \
    -XX:MaxGCPauseMillis=500 \
    -XX:+UnlockExperimentalVMOptions \
    -XX:+ExplicitGCInvokesConcurrent \
    -XX:+UseStringDeduplication \
    -XX:+OptimizeStringConcat \
    -XX:MaxMetaspaceSize=512m \
    -XX:+HeapDumpOnOutOfMemoryError \
    -XX:HeapDumpPath=/var/jenkins/heapdump.hprof \
    -XX:+ExitOnOutOfMemoryError \
    -Djava.awt.headless=true \
    -Dfile.encoding=UTF-8"

  # Key flags explained:
  # -Xms4g              = Initial heap (set == -Xmx to prevent heap resizing
  pauses)
  # -Xmx16g             = Maximum heap — Jenkins can't exceed this
  # -XX:+UseG1GC        = Garbage First collector (recommended for Jenkins,
  JDK 11+)
  # -XX:G1HeapRegionSize= G1 region size (larger = better for big heaps)
  # -XX:MaxGCPauseMillis= Target max GC pause (G1 will try to stay below
  this)
  # -XX:+UseStringDeduplication = G1 feature: deduplicate identical String
  objects (saves memory)
  # -XX:MaxMetaspaceSize= Cap class metadata space (plugin classes live
  here)
  # -XX:+HeapDumpOnOutOfMemoryError = Write heap dump on OOM (for analysis)
  # -XX:+ExitOnOutOfMemoryError = Kill JVM on OOM (K8s will restart it
  cleanly)

  # ── KUBERNETES JENKINS DEPLOYMENT ──────────────────────────────
  # Set JVM heap relative to pod memory limit
  # Use JDK 11+ with container awareness: -XX:+UseContainerSupport (default
  in JDK 11+)
  # JDK automatically reads cgroup memory limits

  resources:
    requests:
      memory: "8Gi"
      cpu: "2"
    limits:
      memory: "16Gi"    # JVM heap capped at ~75% of this = 12GB
      cpu: "4"

  # With UseContainerSupport (default):
  # -XX:MaxRAMPercentage=75.0  → JVM uses 75% of container memory limit as
  max heap
  # This auto-scales: 16GB limit → 12GB heap. No manual -Xmx needed.

  # Full K8s-aware JVM settings:
  export JAVA_OPTS="\
    -XX:+UseContainerSupport \
    -XX:MaxRAMPercentage=75.0 \
    -XX:InitialRAMPercentage=50.0 \
```

```
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=500 \
  -XX:+HeapDumpOnOutOfMemoryError \
  -XX:HeapDumpPath=/var/jenkins/heapdump.hprof \
  -XX:+ExitOnOutOfMemoryError"
```

⚙️ Garbage Collector Selection and Tuning

```
GC Algorithm Comparison for Jenkins:

G1GC (Garbage First) — RECOMMENDED for Jenkins on JDK 11+
   ┌──────────────────────────────────────────────────────────────┐
   │ Design: Regional heap — concurrent marking, incremental GC    │
   │ Behavior: Mixed short pause GCs + occasional full GC          │
   │ Why for Jenkins: Predictable pause targets; handles 4-32GB    │
   │  heaps well; Jenkins has high object allocation from CPS      │
   │  execution and build state deserialization                    │
   │ Tune: -XX:MaxGCPauseMillis (target, not guarantee)            │
   │       -XX:G1HeapRegionSize (default: heap/2048, max 32MB)     │
   │       -XX:G1NewSizePercent (young generation size)            │
   └──────────────────────────────────────────────────────────────┘

ZGC (Z Garbage Collector) — OPTION for JDK 15+, very large heaps
   ┌──────────────────────────────────────────────────────────────┐
   │ Design: Fully concurrent — GC work concurrent with app        │
   │ Behavior: Sub-millisecond pauses (vs G1: hundreds of ms)      │
   │ When to use: Jenkins with 32GB+ heap, UI responsiveness       │
   │  is critical, can afford ~10-20% throughput reduction         │
   │ Enable: -XX:+UseZGC                                           │
   └──────────────────────────────────────────────────────────────┘

Shenandoah — OPTION for JDK 12+, similar to ZGC
  Similar to ZGC in low-pause characteristics
  Available in some OpenJDK distributions (not Oracle JDK)

CMS (Concurrent Mark Sweep) — DEPRECATED (JDK 14+)
  Do not use — removed from JDK 14, was common in older Jenkins setups

Parallel GC — NOT recommended for Jenkins
  Throughput-optimized but high pause times → UI unresponsiveness

Decision tree:
  Heap < 8GB → G1GC (default, no special tuning needed)
  Heap 8-32GB → G1GC with tuned MaxGCPauseMillis
  Heap > 32GB → ZGC or Shenandoah
  JDK < 11 → G1GC (CMS as fallback — but upgrade JDK!)
```

```
# ── GC LOGGING FOR DIAGNOSIS ──────────────────────────────────────
# Always enable GC logging in production — minimal overhead, maximum
insight
JAVA_OPTS="... \
  -
Xlog:gc*:file=/var/log/jenkins/gc.log:time,uptime,level,tags:filecount=5,f
ilesize=50m \
  -XX:+PrintGCDetails \
  -XX:+PrintGCDateStamps"

# Analyze GC logs with:
# gceasy.io (online GC log analyzer)
# GCViewer (open source desktop tool)
# Or: grep 'GC pause' gc.log | awk '{print $NF}' | sort -n | tail -20

# GC pause thresholds that indicate problems:
# < 100ms: excellent (G1 default target)
# 100-500ms: acceptable
# 500ms-2s: noticeable UI lag, investigate
# > 2s: severe — Jenkins UI becomes unresponsive
# Full GC > 5s: critical — Jenkins effectively paused
```

⚙️ Thread Pool Tuning

```
Jenkins uses several thread pools:

1. HTTP Request Thread Pool (Jetty)
   Handles: Web UI requests, REST API calls, webhook deliveries
   Setting: JENKINS_OPTS="--handlerCountMax=300"
   Default: 200 threads
   When to increase: HTTP 503 responses under load, webhook delivery
failures

2. Jenkins Executor Thread Pool
   Handles: Build execution coordination on Controller
   Always set: numExecutors=0 on Controller (don't run builds on
Controller)

3. Remoting Thread Pool
   Handles: Agent connections, file transfers between Controller and
agents
   Thread count tied to: number of connected agents

4. CPS Execution Threads
   Handles: Groovy CPS pipeline execution steps
   Each active pipeline step holds a thread
   Blocking sh steps: thread held until command completes
```

```
# Jetty HTTP thread pool settings
JENKINS_OPTS="\
  --handlerCountMax=300 \
  --handlerCountMaxIdle=20 \
  --httpPort=8080 \
  --prefix=/jenkins"

# For HTTPS termination (recommended via reverse proxy, not Jenkins
directly):
# Use nginx/haproxy in front of Jenkins for SSL termination
# Jenkins speaks HTTP internally, proxy terminates HTTPS externally
```

⚙️ Jenkins System Configuration for Performance

```
// These are set in Manage Jenkins → Configure System
// Or via JCasC

jenkins:
  numExecutors: 0          # Controller NEVER runs builds
                           # All builds on agents — protects Controller
resources
  quietPeriod: 0           # 0 = start immediately on trigger (no
debounce)
                           # Set to 5 for repos with rapid push events
  scmCheckoutRetryCount: 2 # Retry SCM operations on transient failure

// ── SYSTEM MESSAGE ──────────────────────────────────────────────────
// Inform engineers of capacity limits (prevents surprise build failures):
systemMessage: |
  Max concurrent builds: 200 (Kubernetes auto-scaling)
  Build queue wait > 5min: open a ticket for agent capacity increase
  This Jenkins is managed via Configuration as Code.
```

```
# ── JENKINS STARTUP FLAGS ──────────────────────────────────────────
# Disable unused features that consume memory/CPU

# Disable Jenkins usage statistics (phoning home):
-Djenkins.model.Jenkins.slaveAgentPort=-1  # disable fixed JNLP port if
not needed
-Dhudson.model.UsageStatistics.disabled=true

# Disable built-in DNS multibcast (not needed with static agent config):
-Dhudson.DNSMultiCast.disabled=true

# Disable JNLP agent protocol versions (use only latest):
-Djenkins.slaves.DefaultJnlpSlaveReceiver.disabled=true
```

```
# Faster file access on NFS (if JENKINS_HOME is on NFS):
-Dhudson.FilePath.chmod=true

# Reduce CPS execution overhead:
-
Dorg.jenkinsci.plugins.workflow.steps.durable_task.DurableTaskStep.REMOTE_
TIMEOUT=20

# ── JENKINS_HOME FILESYSTEM PERFORMANCE ──────────────────────────
# JENKINS_HOME on network storage (NFS/EFS/Azure Files):
#   - I/O latency kills pipeline durability writes
#   - Use PERFORMANCE_OPTIMIZED durability (fewer writes)
#   - Or: JENKINS_HOME on fast local SSD (EBS gp3 on AWS)

# For Kubernetes Jenkins on EBS:
# storageClass: gp3, iops: 3000, throughput: 125MB/s
# This alone can 2-3x Jenkins responsiveness vs gp2
```

⚙️ Build Queue and Scheduler Tuning

```
Jenkins Build Queue performance:
  The queue scheduler runs frequently to determine which builds to
dispatch
  Performance degrades with:
    - Many label expressions (complex matching = O(n*m) computation)
    - Thousands of queued builds
    - Many agents with complex label sets

Optimization strategies:
  1. Simplify labels: use 'kubernetes' or 'java' not complex expressions
     label 'kubernetes && java && maven && jdk17'
     → 4-way intersection on every scheduling cycle
     Better: label 'maven-jdk17' (single label for common combo)

  2. Limit queue depth:
     Use throttle-concurrent-builds plugin to cap concurrent builds per
job
     Prevent a single fast-committing team from filling the entire queue

  3. Agent capacity matches demand:
     Right-size the Kubernetes cluster autoscaler limits
     Monitor queue wait time — > 60s consistently means capacity is too
low

// Throttle concurrent builds globally and per-job:
pipeline {
    options {
        // Limit: max 5 concurrent builds of THIS specific job
        throttleJobProperty(
            categories: [],
```

```
            throttleEnabled: true,
            throttleOption: 'project',
            maxConcurrentPerNode: 0,
            maxConcurrentTotal: 5
        )
        // OR: disableConcurrentBuilds() for max 1 concurrent
        disableConcurrentBuilds(abortPrevious: true)
    }
}
```

⚙️ Plugin Audit for Performance

```
# Plugins consume heap, CPU, startup time, and thread pool capacity
# Every installed plugin adds overhead even if unused

# Strategy: Quarterly plugin audit
# 1. List all plugins:
java -jar jenkins-cli.jar -s https://jenkins.example.com list-plugins | \
    awk '{print $1}' > installed-plugins.txt

# 2. Find plugins that haven't been used in 90 days:
# (Requires audit trail or usage stats — Jenkins doesn't natively track
per-plugin usage)
# Alternative: grep build logs for plugin step usage

# 3. Test: disable plugin → monitor for 1 week → any build failures?
# Manage Jenkins → Plugin Manager → Installed → uncheck Enabled

# Common performance-heavy plugins to audit:
# - Blue Ocean: if team uses classic UI, Blue Ocean adds overhead with no
benefit
# - Matrix Authorization: simpler than Role Strategy but more memory for
large matrices
# - Build Monitor View: real-time dashboard that polls frequently
# - GitHub Pull Request Builder: can overwhelm GitHub API with polling

# Minimum viable plugin set for Kubernetes-based Jenkins:
# workflow-aggregator (Pipeline)
# git, github
# kubernetes
# credentials, credentials-binding
# role-strategy
# configuration-as-code
# slack OR email-ext
# blueocean OR pipeline-stage-view
# ws-cleanup
# timestamper
```

🔑 Key Concepts

Concept	Meaning
`-Xms` / `-Xmx`	Initial / maximum JVM heap — set equal to prevent resizing pauses
G1GC	Garbage First collector — recommended for Jenkins; regional heap; pause targets
ZGC	Sub-millisecond GC pauses — for very large heaps (32GB+)
`-XX:MaxRAMPercentage`	Container-aware heap sizing — uses % of cgroup memory limit
`-XX:+ExitOnOutOfMemoryError`	JVM exits cleanly on OOM — Kubernetes restarts it
`numExecutors: 0`	Controller runs NO builds — dedicates all resources to coordination
GC pause	STW (Stop-The-World) event where JVM pauses all threads to collect garbage
Metaspace	Non-heap memory for class metadata — grows with plugin count
Thread pool exhaustion	All HTTP threads busy → new requests get 503
Queue scheduler	Jenkins algorithm that matches queued builds to available agents
JENKINS_HOME on SSD	Storage performance directly affects pipeline durability write latency

💬 Short Crisp Interview Answer

> *"Jenkins performance tuning starts with JVM heap sizing — rule of thumb: 4GB for small orgs, 8-16GB for medium, 16-32GB for large. Always set `-Xms` equal to `-Xmx` to prevent heap resize pauses. Use G1GC (`-XX:+UseG1GC`) with a `MaxGCPauseMillis` target of 500ms — G1 handles Jenkins' workload (lots of medium-lived objects from CPS pipeline state) well. In Kubernetes, use `-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0` instead of manual `-Xmx`, so the JVM auto-sizes to 75% of the pod's memory limit. Always set `numExecutors=0` on the Controller — no builds run on the Controller, preserving CPU and heap for coordination. For storage: JENKINS_HOME on an SSD-backed volume (EBS gp3, not gp2) dramatically reduces I/O latency for pipeline state writes. Enable GC logging (`-Xlog:gc*`) and watch for GC pauses over 500ms — that's when UI responsiveness degrades and operators start filing tickets."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Setting `-Xms` much lower than `-Xmx` causes repeated heap expansion pauses.** If `-Xms2g -Xmx16g`, the JVM starts with 2GB and expands to 16GB under load, causing GC pauses at each expansion boundary. Set them equal: `-Xms16g -Xmx16g`. Yes, this means the pod always consumes the full heap allocation — size the memory request accordingly.

- **`-XX:+ExitOnOutOfMemoryError` is critical in Kubernetes.** Without it, an OOM-killed JVM thread causes Jenkins to enter an unstable half-dead state — some operations work, others fail unpredictably. With `ExitOnOutOfMemoryError`, the JVM exits cleanly, Kubernetes restarts the pod, and Jenkins recovers in ~60 seconds. Always add this flag.
- **Metaspace growth with many plugins.** Each plugin loads Java classes into Metaspace (non-heap memory). With 150+ plugins, Metaspace can grow to 1-2GB. If `-XX:MaxMetaspaceSize` is too low, you get `OutOfMemoryError: Metaspace` — a separate error from heap OOM. Set it to 512MB minimum, monitor actual usage.
- **G1GC `MaxGCPauseMillis` is a target, not a guarantee.** G1 tries to stay under the target but will exceed it if the heap is too full or live data is too large for incremental collection. If pauses consistently exceed the target, the heap is too small — increase `-Xmx`.

Topic 8.2 — Build Time Optimization

🔴 Advanced | Making Builds Faster

📌 What It Is — In Simple Terms

Build time directly affects developer feedback loop speed — the time from "I pushed code" to "I know if it's correct." A 5-minute build encourages rapid iteration; a 45-minute build encourages large batch commits and deferred feedback. Build time optimization is the engineering discipline of systematically identifying what's slow in a pipeline and applying targeted fixes: caching, parallelism, incremental compilation, and smarter test strategies.

🔍 Where Build Time Goes — The Anatomy of a Slow Build

```
Typical slow build profile (45 minutes total):
  Stage                          Time    % of total
  ──────────────────────────────────────────────────
  Agent startup / pod creation  30s     1%
  Git checkout                  45s     2%
  Dependency resolution         8min    18%    ← Maven downloading from
internet
  Compilation                   5min    11%    ← Full rebuild, no
incremental
  Unit tests                    12min   27%    ← Sequential, no parallel
  Integration tests             15min   33%    ← Slowest, often redundant
  Docker image build            3min    7%     ← Re-downloading base
layers
  Push to registry              1min    2%
  ──────────────────────────────────────────────────
  Total                         45min   100%

After optimization (target: < 10 minutes):
  Agent startup / pod creation  25s     4%     ← Pre-warmed pod pools
```

```
  Git checkout (shallow)        15s     3%     ← Shallow clone: depth=1
  Dependency resolution         30s     5%     ← Maven cache PVC mounted
  Compilation (incremental)     2min    20%    ← Incremental compiler
  Unit tests (parallel)         3min    30%    ← 4 parallel test forks
  Integration tests (selective) 2min    20%    ← Only tests for changed
code
  Docker image build (cached)   1min    10%    ← Layer cache hit
  Push to registry              30s     5%     ← Only changed layers
pushed
  ──────────────────────────────────────────────────
  Total                         ~10min  100%   ← 4.5x improvement
```

⚙️ Caching Strategies

```
// ── MAVEN DEPENDENCY CACHE (Kubernetes PVC) ──────────────────────
// Without cache: Maven downloads ~200MB on every build
// With cache: Cache hit → ~10-30s to read local repo vs 8+ minutes to
download

// In pod template YAML:
// volumes:
// - name: maven-cache
//   persistentVolumeClaim:
//     claimName: maven-cache-pvc   # ReadWriteMany PVC (EFS/NFS)
// containers:
// - name: maven
//   volumeMounts:
//   - name: maven-cache
//     mountPath: /root/.m2/repository

// Pre-create the PVC:
// kubectl create -f - <<EOF
// apiVersion: v1
// kind: PersistentVolumeClaim
// metadata:
//   name: maven-cache-pvc
//   namespace: jenkins-builds
// spec:
//   accessModes: [ReadWriteMany]     # Multiple pods can read/write
simultaneously
//   storageClassName: efs-sc         # AWS EFS for ReadWriteMany
//   resources:
//     requests:
//       storage: 50Gi
// EOF

// In pipeline:
stage('Build') {
    steps {
        container('maven') {
```

```
            sh """
                mvn clean package \
                    -Dmaven.repo.local=/root/.m2/repository \  # Explicit
path
                    --no-transfer-progress \                    # Suppress
download noise
                    -T 4 \                                      # 4
threads for compilation
                    -B                                          # Batch
mode (no ANSI)
            """
        }
    }
}

// ── NPM / NODE_MODULES CACHE ──────────────────────────────────────
// volumes:
// - name: npm-cache
//   persistentVolumeClaim:
//     claimName: npm-cache-pvc

stage('Install Dependencies') {
    steps {
        container('node') {
            sh """
                # Use npm ci with cache directory on PVC
                npm ci \
                    --cache /npm-cache \
                    --prefer-offline \
                    2>&1 | tail -5   # Only show last 5 lines (suppress
download noise)
            """
        }
    }
}

// ── GRADLE CACHE ──────────────────────────────────────────────────
// Mount: /root/.gradle on PVC
// In Gradle build:
// org.gradle.caching=true
// org.gradle.daemon=false    # Daemons don't work well in ephemeral
containers
// org.gradle.parallel=true
// org.gradle.workers.max=4
```

⚙️ Parallel Stage Execution

```
// ── PARALLEL TEST EXECUTION ───────────────────────────────────────
pipeline {
    agent {
```

```
        kubernetes {
            yaml '''
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: maven
    image: maven:3.9-eclipse-temurin-17
    command: ["cat"]
    tty: true
    resources:
      requests: {cpu: "2", memory: "4Gi"}
      limits:   {cpu: "4", memory: "8Gi"}
'''
        }
    }

    stages {
        stage('Build') {
            steps {
                container('maven') {
                    sh 'mvn clean package -DskipTests -T 4'
                }
            }
        }

        // Run different test suites in parallel — each gets full CPU
        stage('Test') {
            parallel {
                stage('Unit Tests') {
                    steps {
                        container('maven') {
                            sh 'mvn test -pl module-core -pl module-auth'
                        }
                    }
                    post {
                        always {
                            junit 'module-*/target/surefire-reports/*.xml'
                        }
                    }
                }

                stage('Integration Tests') {
                    steps {
                        container('maven') {
                            sh 'mvn verify -pl module-integration -
DskipUnitTests'
                        }
                    }
                    post {
                        always {
                            junit 'module-integration/target/failsafe-
reports/*.xml'
                        }
```

```
                    }
                }

                stage('Contract Tests') {
                    steps {
                        container('maven') {
                            sh 'mvn test -pl module-contracts'
                        }
                    }
                }
            }
        }
    }
}

// ── DYNAMIC PARALLEL ACROSS SERVICES (Monorepo) ──────────────────
stage('Test All Services') {
    steps {
        script {
            // Build a map of parallel stages — one per service
            def services = ['auth', 'payments', 'orders', 'inventory']

            def parallelStages = services.collectEntries { svc ->
                ["Test ${svc}": {
                    container('maven') {
                        sh "mvn test -pl services/${svc}"
                    }
                }]
            }

            // Execute all in parallel with failFast — one failure stops
all
            parallelStages.failFast = true
            parallel parallelStages
        }
    }
}
```

⚙️ Incremental Builds — Only Rebuild What Changed

```
// ── DETECTING CHANGED FILES (Monorepo / Multi-module) ─────────────
stage('Detect Changes') {
    steps {
        script {
            // Get list of changed files since last successful build
            def changedFiles = sh(
                script: """
                    git diff --name-only \
                        ${env.GIT_PREVIOUS_SUCCESSFUL_COMMIT ?:
'HEAD~1'}..HEAD
```

```
                """,
                returnStdout: true
            ).trim().split('\n') as List

            echo "Changed files: ${changedFiles}"

            // Determine which services need rebuilding
            env.BUILD_AUTH     = changedFiles.any {
it.startsWith('services/auth/') }.toString()
            env.BUILD_PAYMENTS = changedFiles.any {
it.startsWith('services/payments/') }.toString()
            env.BUILD_SHARED   = changedFiles.any {
it.startsWith('shared/') }.toString()

            // If shared/ changed — rebuild everything
            if (env.BUILD_SHARED == 'true') {
                env.BUILD_AUTH     = 'true'
                env.BUILD_PAYMENTS = 'true'
            }

            echo "Services to build: auth=${env.BUILD_AUTH}
payments=${env.BUILD_PAYMENTS}"
        }
    }
}

stage('Build Services') {
    parallel {
        stage('Auth Service') {
            when { expression { return env.BUILD_AUTH == 'true' } }
            steps {
                sh 'mvn package -pl services/auth --also-make'
            }
        }

        stage('Payments Service') {
            when { expression { return env.BUILD_PAYMENTS == 'true' } }
            steps {
                sh 'mvn package -pl services/payments --also-make'
            }
        }
    }
}

// ── MAVEN MULTI-MODULE: --also-make --also-make-dependents ──────────
// --also-make (-am): also build upstream modules (modules this depends
on)
// --also-make-dependents (-amd): also build downstream modules (modules
that depend on this)

// Only build the auth module AND modules it depends on:
sh 'mvn package -pl services/auth --also-make'
```

```
// Build payments AND everything that depends on payments:
sh 'mvn package -pl services/payments --also-make-dependents'
```

⚙️ Docker Layer Cache Optimization

```
# ── LAYER-ORDERED DOCKERFILE FOR MAXIMUM CACHE HITS ────────────────
# Order: most stable → most volatile (least to most frequently changing)

FROM eclipse-temurin:17-jdk-alpine AS builder

WORKDIR /app

# Layer 1: System deps (rarely changes — good cache hit)
RUN apk add --no-cache curl

# Layer 2: Build tool config (changes when Maven version/config changes)
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./

# Layer 3: Dependencies (changes when pom.xml changes — not on every
commit)
# This is the critical optimization: download deps in a separate layer
RUN ./mvnw dependency:go-offline -B

# Layer 4: Source code (changes on every commit — always a cache miss
here)
COPY src/ ./src/

# Layer 5: Compile (always runs if source changed)
RUN ./mvnw package -DskipTests -B
```

```
// Kaniko with caching in pipeline:
stage('Build Image') {
    steps {
        container('kaniko') {
            sh """
                /kaniko/executor \
                    --context=dir:///workspace \
                    --dockerfile=/workspace/Dockerfile \
                    --destination=${IMAGE_NAME}:${IMAGE_TAG} \
                    --cache=true \
                    --cache-repo=${REGISTRY}/cache/myapp \
                    --cache-ttl=168h \
                    --snapshot-mode=redo \
                    --use-new-run
            """
        }
    }
```

```
}
// --cache=true: push/pull intermediate layers to cache repo
// --cache-repo: where to store the cache
// --cache-ttl: how long to keep cached layers (168h = 1 week)
// --snapshot-mode=redo: more reliable cache invalidation
```

⚙️ Test Optimization Strategies

```
// ── TEST PARALLELISM WITHIN A SINGLE BUILD ──────────────────────────
// Maven Surefire: run tests in parallel
stage('Unit Tests') {
    steps {
        sh """
            mvn test \
                -Dsurefire.useFile=false \
                -Dsurefire.failIfNoSpecifiedTests=false \
                -Dsurefire.parallel=methods \
                -Dsurefire.perCoreThreadCount=true \
                -Dsurefire.threadCount=2 \
                -Dsurefire.forkCount=4 \
                -Dsurefire.reuseForks=true \
                -B
        """
        // forkCount=4: 4 JVM processes running tests in parallel
        // threadCount=2: 2 threads per JVM = 8 concurrent test threads
total
    }
}

// ── TEST SPLITTING ACROSS MULTIPLE PODS ─────────────────────────────
// For very large test suites: split tests across N pods, run in parallel
// Requires a test splitting strategy

pipeline {
    agent none
    stages {
        stage('Test') {
            steps {
                script {
                    // Discover test classes
                    def testClasses = sh(
                        script: "find . -name '*Test.java' | sed
's|.*/||;s|.java||'",
                        returnStdout: true
                    ).trim().split('\n')

                    // Split into 4 groups
                    def chunkSize = (int) Math.ceil(testClasses.size() /
4.0)
                    def chunks    =
```

```
testClasses.toList().collate(chunkSize)

                    // Run each chunk in a separate pod in parallel
                    def parallelBranches = [:]
                    chunks.eachWithIndex { chunk, i ->
                        def chunkList = chunk.join(',')
                        def idx       = i
                        parallelBranches["Test Shard ${idx + 1}"] = {
                            node('kubernetes') {
                                checkout scm
                                sh "mvn test -Dtest=${chunkList} -B"
                                junit 'target/surefire-reports/*.xml'
                            }
                        }
                    }
                    parallel parallelBranches
                }
            }
        }
    }
}

// ── FLAKY TEST HANDLING ──────────────────────────────────────────────
// Retry flaky tests automatically before failing
stage('Tests with Retry') {
    steps {
        retry(3) {
            sh """
                mvn test \
                    -Dsurefire.rerunFailingTestsCount=2 \
                    -B
                # Surefire rerunFailingTestsCount: auto-rerun failed tests
up to N times
                # If test passes on retry → marked as "flaky" not "failed"
            """
        }
    }
}
```

⚙️ Shallow Clone and Sparse Checkout Optimization

```
// Shallow clone: only fetch the most recent commits (not full history)
// For CI: you almost never need the full Git history
// Speedup: varies 10x-100x for large repos with long history

checkout([
    $class: 'GitSCM',
    branches: [[name: env.BRANCH_NAME]],
    extensions: [
        // Depth 1: only the latest commit (fastest)
```

```
        // Depth > 1: needed for git diff against N commits back
        [$class: 'CloneOption', shallow: true, depth: 2, timeout: 10],

        // Sparse checkout: only fetch directories relevant to this build
        // Critical for monorepos — auth service doesn't need analytics
code
        [$class: 'SparseCheckoutPaths', sparseCheckoutPaths: [
            [path: 'services/auth/'],
            [path: 'shared/libs/'],
            [path: 'deploy/helm/']
        ]],

        // Only fetch the branch being built (not all branches)
        [$class: 'PruneStaleBranch']
    ],
    userRemoteConfigs: [[
        url: 'https://github.com/myorg/myrepo.git',
        credentialsId: 'github-creds'
    ]]
])
// For a 5GB monorepo with 5 years of history:
// Full clone:   45-120 seconds
// Shallow+sparse: 5-15 seconds → significant improvement
```

🔑 Key Concepts

Concept	Meaning
`-T 4`	Maven parallel thread count — compile N modules simultaneously
`dependency:go-offline`	Pre-download all Maven deps — exploits layer cache
PVC for cache	Persistent Volume Claim mounted in build pods — survives pod deletion
ReadWriteMany	PVC access mode allowing multiple pods to use same cache volume
`forkCount` / `threadCount`	Surefire parallel test execution — N JVM processes × M threads
Sparse checkout	Fetch only specific directories — critical for monorepo performance
Shallow clone	Fetch only recent commits — eliminates full history download
`failFast: true`	In parallel stages — first failure aborts all sibling stages
Layer-ordered Dockerfile	Most stable layers first — maximizes Docker layer cache hits
Incremental build	Only rebuild modules whose source files changed
`--also-make`	Maven: build specified module AND all its upstream dependencies

---

💬 Short Crisp Interview Answer

> *"Build time optimization has four levers: caching, parallelism, incrementality, and Git efficiency. Caching: mount a PVC into build pods for Maven/npm/Gradle caches — without this, every build downloads 200MB+ of dependencies from the network. Parallelism: split test suites into parallel stages (unit, integration, contract tests in parallel), and within Maven use `-T 4` for parallel module compilation and Surefire `forkCount=4` for parallel test execution. Incrementality: in monorepos, detect changed files with `git diff` and only build services with actual changes using Maven's `--also-make` flag. Git efficiency: shallow clone (`depth: 1`) eliminates downloading full history; sparse checkout fetches only the directories relevant to the service being built — for a 5GB monorepo this alone reduces checkout from 90 seconds to 10. Docker layer caching: order Dockerfile layers from most stable (base image, system deps) to most volatile (source code), so the expensive `mvn dependency:go-offline` layer is cached and only rebuilt when pom.xml changes."*

⚠ Tricky Edge Cases & Gotchas

- **⚠ ReadWriteMany PVC causes cache corruption under heavy concurrent writes.** Multiple build pods writing to the same Maven local repository simultaneously can corrupt the cache (partially downloaded JARs, corrupt checksums). Mitigation: use a proxy cache (Nexus/Artifactory) instead of a shared PVC for dependencies — each pod downloads from the fast internal registry. Or: accept occasional cache corruption and add `-U` (force update snapshots) to the Maven command.
- **`failFast: true` in parallel stages aborts sibling stages mid-execution.** If unit tests fail with `failFast: true`, running integration tests are aborted. This is usually correct for CI (fast failure feedback). But if the integration test has already started a database migration that gets aborted mid-way, you may need cleanup logic in `post { always }`.
- **Shallow clone breaks `git describe` and some changelog tools.** Tools that use `git describe --tags` to generate version numbers need tag history. With `depth: 1`, only the latest commit is available — `git describe` fails with "fatal: No tags can describe". Use `depth: 100` or `noTags: false` to include tag references, or generate version numbers from BUILD_NUMBER instead.
- **Maven `-T` flag with non-thread-safe plugins.** Some Maven plugins are not thread-safe and fail with parallel module builds (`-T`). The error is usually a race condition or shared file write conflict. If you see intermittent failures with `-T 4`, try `-T 1` (serial) to confirm it's a thread-safety issue, then identify and isolate the offending plugin.

Topic 8.3 — Ephemeral vs Persistent Agents ⚠

🟡 Intermediate | The Build Infrastructure Architecture Decision

---

📌 What It Is — In Simple Terms

Every Jenkins build runs on an **agent** (also called a node or executor). The fundamental architectural choice is whether that agent persists between builds (persistent agent) or is created fresh for each build and destroyed when done (ephemeral agent). This decision affects build isolation, security, cost, performance, and operational complexity in ways that interviewers probe deeply — it reveals whether you understand CI/CD infrastructure at an architectural level.

⚙️ Persistent Agents — Characteristics

```
Persistent agents: VMs or bare-metal machines that stay running
indefinitely
  Configured once, run many builds over their lifetime
  State accumulates: build artifacts, caches, tool installations, log
files

Architecture:
  Jenkins Controller ←────── Persistent Agent (always connected)
                     (SSH or JNLP, reconnects after restart)

Examples:
  - AWS EC2 instance running permanently
  - On-premise VM in VMware cluster
  - Dedicated Mac mini for iOS builds
  - Bare-metal server with GPU for ML model training

Characteristics:
  ✅ Fast build start: agent already connected, no startup time
  ✅ Cache warm: Maven/npm cache from previous build still on disk
  ✅ Specialized hardware: GPU, FPGA, hardware security modules (HSMs)
  ✅ Simple to reason about: what you see on the agent = what builds see
  ✅ Works offline: no K8s/cloud dependency

  ❌ Dirty state: build N leaves files that affect build N+1
  ❌ "Works on my agent" bugs: unique state not reproducible elsewhere
  ❌ Under-utilization: agent idle 70% of time at non-peak hours
  ❌ Over-utilization at peak: builds queue waiting for agents
  ❌ Maintenance: OS patches, tool upgrades, disk cleanup — manual ops
  ❌ Security: compromised agent persists — all future builds affected
  ❌ Cost: paying for EC2 even when not building
  ❌ Snowflake agents: each agent drifts over time — hard to reproduce
```

⚙️ Ephemeral Agents — Characteristics

```
Ephemeral agents: Created for one build, destroyed when done
  Kubernetes pod, Docker container, AWS EC2 Spot instance (started on
demand)
  Each build starts with a known, clean state
  State does NOT accumulate between builds

Architecture:
  Jenkins Controller → (schedule build) → Kubernetes API
  Kubernetes → create pod (agent container + build containers)
  Pod connects to Jenkins Controller as agent
  Build runs in pod
  Pod terminates → state gone
```

```
  Next build → new pod → clean state

Characteristics:
  ✅ Clean state: every build starts identical
  ✅ Reproducible: any failure can be reproduced (same image, same env)
  ✅ Elastic: 0 agents at night, 100 at peak, autoscaled
  ✅ Cost: pay only for build time (spot instances: 70% cheaper)
  ✅ Security: compromised build pod → deleted at end of build → contained
  ✅ No maintenance: pod spec in Git = reproducible, versionable,
reviewable
  ✅ Isolation: builds can't interfere with each other

  ❌ Cold start latency: pod creation takes 30-90 seconds
  ❌ No persistent cache: Maven downloads all deps every build (mitigate
with PVC)
  ❌ Kubernetes dependency: needs K8s cluster to run on
  ❌ Complex networking: Controller → K8s API → pod networking
  ❌ Harder to debug: pod gone after build → must inspect logs before
deletion
  ❌ Special hardware: GPUs, Mac hardware not easily available as K8s pods
```

⚙️ Side-by-Side Comparison

```
┌──────────────────────────┬──────────────────────────┬──────────────────────
──────┐
│ Dimension                │ Persistent Agent         │ Ephemeral Agent
│
├──────────────────────────┼──────────────────────────┼──────────────────────
──────┤
│ Build isolation          │ ❌ Shared state          │ ✅ Perfect
isolation      │
│ Reproducibility          │ ❌ Snowflake risk        │ ✅ Same image
every build │
│ Build start time         │ ✅ ~1 second             │ ⚠ 30-90s pod
startup    │
│ Dependency caching       │ ✅ On-disk from prev build│ ⚠ Requires PVC
setup    │
│ Cost (light load)        │ ❌ Idle EC2 costs money  │ ✅ Zero cost at
idle      │
│ Cost (peak load)         │ ❌ Fixed ceiling         │ ✅ Scales to
demand       │
│ Specialized hardware     │ ✅ GPU, Mac, FPGA        │ ❌ Limited (cloud
GPU OK) │
│ Security blast radius    │ ❌ Compromise persists   │ ✅ Destroyed
post-build   │
│ Maintenance burden       │ ❌ OS patches, disk clean │ ✅ Update image
tag        │
│ Network complexity       │ ✅ Simple SSH/JNLP       │ ❌ K8s networking
│
│ Observability            │ ✅ SSH in anytime        │ ⚠ Must retain
```

```
logs        │
│ Offline capability        │ ✅ No cloud dependency    │ ❌ Needs
K8s/cloud        │
└──────────────────────────┴──────────────────────────┴──────────────────
──────┘
```

⚙️ Hybrid Strategy — The Production Reality

```
Most production Jenkins installations use BOTH types:

Ephemeral (Kubernetes pods) for:
  - All standard application builds (Java, Node, Python, Go)
  - Docker image builds (Kaniko)
  - Terraform/Helm deployments
  - Security scans
  → 95% of builds

Persistent agents for:
  - iOS/macOS builds (Mac mini fleet — Apple hardware required)
  - Android emulator tests (require full OS with KVM support)
  - Large ML model training (GPU instances, expensive setup time)
  - Hardware-in-the-loop testing (physical device required)
  - Air-gapped environments (no internet, no K8s API)
  → 5% of builds, specialized use cases
```

```yaml
# JCasC: Configure both types

jenkins:
  clouds:
    # Ephemeral: Kubernetes pods (primary)
    - kubernetes:
        name: "kubernetes"
        serverUrl: ""
        namespace: "jenkins-builds"
        containerCapStr: "100"
        templates:
          - name: "default"
            containers:
              - name: jnlp
                image: "jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-
jdk17"

  # Persistent: Static agents for specialized workloads
  nodes:
    - permanent:
        name: "mac-mini-1"
        remoteFS: "/Users/jenkins/workspace"
        numExecutors: 2
```

```
        labelString: "macos ios xcode"
        launcher:
          ssh:
            host: "mac-mini-1.internal.example.com"
            port: 22
            credentialsId: "mac-mini-ssh-key"
            launchTimeoutSeconds: 60
            maxNumRetries: 3
        retentionStrategy:
          always:    # Always keep this agent connected

    - permanent:
        name: "gpu-agent-1"
        remoteFS: "/home/jenkins/workspace"
        numExecutors: 1
        labelString: "gpu ml pytorch cuda"
        launcher:
          ssh:
            host: "gpu-server-1.internal.example.com"
            port: 22
            credentialsId: "gpu-agent-ssh-key"
```

⚙️ Mitigating Ephemeral Agent Cold Start Latency

```
// Problem: 30-90 second pod startup time delays builds

// Solution 1: Pod pre-warming (Kubernetes standby pods)
// Keep N "warm" pods in Ready state, waiting for work
// When a build starts, it immediately gets a warm pod
// A new pod is created to replace the consumed warm pod

// In pod template JCasC:
// idleMinutes: 10  ← Keep idle pods alive for 10 minutes
// This only works if builds arrive frequently enough to justify the cost

// Solution 2: Reduce image size → faster pull
// Smaller images = faster container startup
// alpine-based images: 50-200MB vs debian: 200-500MB
// Pre-pull images to nodes: DaemonSet that pulls your common build images

// Solution 3: Node pre-warming (cluster autoscaler pre-scaling)
// Configure cluster autoscaler to maintain N "over-provisioned" nodes
// New pods schedule onto waiting nodes immediately instead of waiting for
node creation

// Solution 4: BuildKit/Kaniko with image cache
// First pull of kaniko:latest can be slow (250MB image)
// Use a mirror registry inside the cluster that caches images
// Subsequent pulls from local registry: 2-5x faster
```

```
// Solution 5: Accept and design around it
// 30-60s startup is fine for most builds
// If total build is 10 minutes, 60s startup = 10% overhead = acceptable
// Only optimize if build time is already fast enough that startup
dominates

// Measuring actual startup time:
pipeline {
    agent { kubernetes { ... } }
    stages {
        stage('Record Agent Ready Time') {
            steps {
                script {
                    // BUILD_TIMESTAMP set by Jenkins at queue time
                    // Current time = agent ready time
                    def queuedAt   = currentBuild.startTimeInMillis
                    def agentReady = System.currentTimeMillis()
                    def startupMs  = agentReady - queuedAt
                    echo "Agent startup time: ${startupMs}ms"
                    // Feed to Prometheus for trending
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Ephemeral agent	Created per-build, destroyed after — clean state every time
Persistent agent	Long-running, connected continuously — state accumulates
Snowflake agent	Persistent agent that drifted — unique state, hard to reproduce
Cold start	Time for new pod/instance to be ready for work
Pod pre-warming	Maintain idle ready pods to reduce cold start latency
`idleMinutes`	How long to keep an idle pod alive before deleting
ReadWriteMany PVC	Cache volume shared across many build pods simultaneously
Spot instances	Cloud preemptible VMs for ephemeral agents — 70% cost savings
Hybrid architecture	Ephemeral for standard builds, persistent for specialized hardware
`numExecutors: 2`	Persistent agent can run 2 builds concurrently

💬 Short Crisp Interview Answer

> *"Ephemeral agents (Kubernetes pods created per build, deleted after) and persistent agents (long-running VMs) each have a specific role. Ephemeral wins on isolation, security, cost efficiency, and reproducibility — each build gets an identical clean environment, a compromised pod is deleted at build end, and you pay only for actual build time. Persistent wins on cold start speed (already connected), warm disk caches, and specialized hardware requirements (Mac hardware for iOS builds, GPUs, hardware-in-the-loop testing). In production, the answer is hybrid: Kubernetes pod agents for 95% of builds — all standard application CI, Docker builds, deployments — with a dedicated Mac mini fleet for iOS and GPU nodes for ML. The cold start penalty for ephemeral agents (30-90 seconds) is acceptable when total build time is measured in minutes. Mitigate it with `idleMinutes` on pod templates to keep warm pods available, and by mounting a ReadWriteMany PVC for Maven/npm caches so the dependency download penalty is eliminated."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **"Works on the agent" bugs are the #1 argument for ephemeral.** Persistent agents accumulate state: a developer's build leaves a compiled artifact; a subsequent build skips compilation because the artifact "already exists." The build passes locally but fails in production because the artifact was from a different commit. Ephemeral agents make this class of bug impossible — the workspace is always clean.
- **Ephemeral agents and `stash` / `unstash` performance.** When parallel stages each run in separate pods, files must be transferred via `stash` (pod → Controller → pod). For large artifacts (compiled JARs, test results), this is slow and uses Controller memory. Alternative: use a shared PVC or push artifacts to Nexus between stages.
- **Persistent agents need regular cleanup pipelines.** A persistent agent accumulates build artifacts, log files, and stale workspaces. Without a cleanup job, the disk fills and builds fail. Schedule: `sh 'find /home/jenkins/workspace -mtime +7 -type d -maxdepth 1 | xargs rm -rf'` on each agent daily.
- **Persistent agent tool version drift.** Over time, different persistent agents get slightly different versions of Java, Maven, or Docker — because updates were applied manually and not applied to all agents simultaneously. This causes "works on agent-1, fails on agent-2" bugs. Solve with: regular re-imaging of persistent agents from Ansible/Packer templates, not manual installation.

Topic 8.4 — Jenkins Backup and Recovery

🟡 Intermediate | Protecting the CI Platform

📌 What It Is — In Simple Terms

Jenkins backup and recovery is the operational practice of protecting Jenkins data — job configurations, build history, credentials, plugin installations, system configuration — so that when the Jenkins Controller fails (disk corruption, accidental deletion, catastrophic upgrade gone wrong), you can restore it to a functional state within an acceptable time. For most engineering organizations, Jenkins is tier-1 infrastructure. A day without CI means hundreds of engineers can't merge code safely.

🔍 What Needs to Be Backed Up

```
JENKINS_HOME structure and what each part contains:

/var/jenkins_home/
├── config.xml                  ← Jenkins system configuration
│                                  (security, global settings, tools)
│                                  ← CRITICAL: restore this → system
config restored
│
├── credentials.xml             ← All global credentials (AES-256
encrypted)
│                                  ← CRITICAL: without this, all secrets
lost
│
├── secrets/                    ← Master encryption key for
credentials.xml
│   ├── master.key              ← WITHOUT THIS: credentials.xml unreadable
│   ├── hudson.util.Secret      ← Additional secret material
│   └── org.jenkinsci.main.*.key
│                                  ← CRITICAL: backup alongside
credentials.xml
│
├── jobs/                       ← All job configurations + build history
│   ├── my-pipeline/
│   │   ├── config.xml          ← Job configuration
│   │   └── builds/             ← Build history, logs, artifacts
│   │       ├── 1/              ← Build #1
│   │       │   ├── log         ← Console output
│   │       │   ├── build.xml   ← Build metadata
│   │       │   └── archive/    ← Archived artifacts
│   │       └── ...
│
├── plugins/                    ← Installed plugin files (.jpi/.hpi)
│                                  ← Backup: optional if you have
plugins.txt
│
├── nodes/                      ← Static agent configurations
│                                  ← Restore: agents reconnect
automatically
│
├── users/                      ← Local user accounts (if not using LDAP)
│                                  ← CRITICAL if using local auth
│
├── logs/                       ← Jenkins system logs
│                                  ← Don't back up — regenerates
│
└── workspace/                  ← Active build workspaces
                                   ← Don't back up — transient build data

Recovery priority (what to restore first):
  1. secrets/ (master key) — MUST restore before credentials
```

```
  2. credentials.xml + secrets/ (together, atomically)
  3. config.xml (system configuration)
  4. plugins/ or plugins.txt (plugin installations)
  5. jobs/ — job definitions (config.xml per job)
  6. jobs/ build history (optional — often not worth restoring)
```

⚙️ Backup Strategy 1: JCasC + Git (Configuration-as-Code)

```
THIS IS THE MODERN APPROACH — most of Jenkins config is in Git

What JCasC + Git covers:
  ✅ All system configuration (config.xml equivalent) — in jenkins.yaml
  ✅ All credentials definitions (referencing env vars for values)
  ✅ All plugin configurations
  ✅ Cloud/agent configurations
  ✅ All job definitions via Organization Folder auto-discovery
  ✅ All Shared Library configurations

What still needs traditional backup:
  ⚠️ secrets/ directory — the master encryption key (not in Git —
sensitive)
  ⚠️ credentials.xml — if you use static Jenkins credentials (not Vault)
  ⚠️ Build history — if you need to preserve past build records
  ⚠️ User accounts (if not using LDAP/SSO)

Recovery with JCasC:
  Time: 3-10 minutes (vs hours with traditional backup)
  Process:
    1. Start fresh Jenkins container (same version)
    2. Restore secrets/ directory from secure backup (AWS Secrets Manager,
Vault)
    3. Apply jenkins.yaml (JCasC) → full system config restored
    4. Organization Folders auto-discover all repos → all pipelines
recreated
    5. Jenkins fully functional

This is the fastest recovery path and the main reason JCasC adoption
is an operational win beyond just code review of config.
```

⚙️ Backup Strategy 2: Filesystem Backup (Traditional)

```bash
#!/bin/bash
# jenkins-backup.sh — Run nightly via cron or scheduled Jenkins job

JENKINS_HOME="/var/jenkins_home"
BACKUP_DEST="s3://my-jenkins-backups/$(date +%Y/%m/%d)"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
```

```
BACKUP_FILE="/tmp/jenkins-backup-${TIMESTAMP}.tar.gz"

echo "Starting Jenkins backup at $(date)"

# ── WHAT TO INCLUDE ──────────────────────────────────────────────
tar czf "${BACKUP_FILE}" \
    --exclude="${JENKINS_HOME}/workspace"     \  # Exclude: transient
build workspaces
    --exclude="${JENKINS_HOME}/logs"          \  # Exclude: system logs
    --exclude="${JENKINS_HOME}/cache"         \  # Exclude: tool download
caches
    --exclude="${JENKINS_HOME}/fingerprints"  \  # Exclude: large artifact
fingerprints
    --exclude="${JENKINS_HOME}/jobs/*/builds/*/archive" \  # Exclude:
build artifacts (large!)
    --exclude="*.tmp"                         \
    -C "$(dirname ${JENKINS_HOME})" \
    "$(basename ${JENKINS_HOME})"

BACKUP_SIZE=$(du -sh "${BACKUP_FILE}" | cut -f1)
echo "Backup size: ${BACKUP_SIZE}"

# ── UPLOAD TO S3 ─────────────────────────────────────────────────
aws s3 cp "${BACKUP_FILE}" "${BACKUP_DEST}/jenkins-
backup-${TIMESTAMP}.tar.gz" \
    --storage-class STANDARD_IA \
    --server-side-encryption AES256

# ── VERIFY UPLOAD ────────────────────────────────────────────────
aws s3 ls "${BACKUP_DEST}/" | grep "${TIMESTAMP}"

# ── RETENTION POLICY: Keep 30 days ───────────────────────────────
aws s3 ls s3://my-jenkins-backups/ | \
    awk '{print $4}' | \
    sort | head -n -30 | \
    xargs -I{} aws s3 rm "s3://my-jenkins-backups/{}"

# ── CLEANUP LOCAL TEMP FILE ──────────────────────────────────────
rm -f "${BACKUP_FILE}"

echo "Backup complete: ${BACKUP_DEST}/jenkins-backup-${TIMESTAMP}.tar.gz"
```

```
// Run backup as a Jenkins Pipeline job (backs up Jenkins from within
Jenkins!)
pipeline {
    agent { label 'kubernetes' }

    triggers {
        cron('H 2 * * *')   // 2am daily, H = spread across the hour
    }
```

```
    stages {
        stage('Backup Jenkins') {
            steps {
                withCredentials([
                    file(credentialsId: 'jenkins-backup-kubeconfig',
variable: 'KUBECONFIG'),
                    string(credentialsId: 'jenkins-backup-s3-bucket',
variable: 'BUCKET')
                ]) {
                    sh """
                        # Copy JENKINS_HOME from the Jenkins pod to S3
                        TIMESTAMP=\$(date +%Y%m%d_%H%M%S)
                        JENKINS_POD=\$(kubectl get pods -n jenkins -l
app=jenkins \
                            -o jsonpath='{.items[0].metadata.name}')

                        # Create backup inside the pod
                        kubectl exec \$JENKINS_POD -n jenkins -- \
                            tar czf /tmp/jenkins-backup.tar.gz \
                            --exclude=/var/jenkins_home/workspace \
                            --exclude=/var/jenkins_home/logs \
                            --exclude=/var/jenkins_home/cache \
                            /var/jenkins_home

                        # Copy from pod to local
                        kubectl cp jenkins/\$JENKINS_POD:/tmp/jenkins-
backup.tar.gz \
                            /tmp/jenkins-backup-\$TIMESTAMP.tar.gz

                        # Upload to S3
                        aws s3 cp /tmp/jenkins-backup-\$TIMESTAMP.tar.gz \
                            s3://\$BUCKET/\$TIMESTAMP/jenkins-
backup.tar.gz

                        # Cleanup
                        kubectl exec \$JENKINS_POD -n jenkins -- \
                            rm /tmp/jenkins-backup.tar.gz
                        rm /tmp/jenkins-backup-\$TIMESTAMP.tar.gz

                        echo "Backup complete: s3://\$BUCKET/\$TIMESTAMP/"
                    """
                }
            }
        }
    }

    post {
        failure {
            slackSend channel: '#ops-alerts', color: 'danger',
                      message: "🔴 Jenkins backup FAILED — manual backup
required immediately!"
        }
    }
}
```

⚙️ Backup Strategy 3: Kubernetes PVC Snapshot

```
# For Jenkins running on Kubernetes with PVC for JENKINS_HOME
# Use Kubernetes VolumeSnapshot (CSI driver required)

# ── CREATE SNAPSHOT (AWS EBS CSI) ──────────────────────────────
cat <<EOF | kubectl apply -f -
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: jenkins-home-snapshot-$(date +%Y%m%d)
  namespace: jenkins
spec:
  volumeSnapshotClassName: csi-aws-vsc
  source:
    persistentVolumeClaimName: jenkins-home-pvc
EOF

# ── LIST SNAPSHOTS ─────────────────────────────────────────────
kubectl get volumesnapshots -n jenkins

# ── RESTORE FROM SNAPSHOT ──────────────────────────────────────
# Create new PVC from snapshot:
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins-home-restored
  namespace: jenkins
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources:
    requests:
      storage: 100Gi
  dataSource:
    name: jenkins-home-snapshot-20240115
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
EOF

# Update Jenkins StatefulSet to use jenkins-home-restored PVC
# Scale down Jenkins → update PVC reference → scale up
```

⚙️ Thin Backup Plugin (ThinBackup)

```
Plugin ID: thinBackup
What it does:
  - Scheduled backups of JENKINS_HOME within Jenkins
  - Incremental backups (only changed files since last backup)
  - Full backup + differential backup strategy
  - Configurable retention policy

Configuration (JCasC):
unclassified:
  thinBackup:
    fullBackupSchedule: "H 2 * * 7"     # Full backup weekly (Sunday 2am)
    diffBackupSchedule: "H 2 * * 1-6"   # Incremental backup daily
    backupPath: "/jenkins-backup"        # Path to backup directory
(should be on separate volume)
    nrMaxStoredFull: 4                   # Keep 4 weekly full backups
    excludedFilesRegex: "workspace|logs|cache|fingerprints"
    cleanUpDiff: true
    moveOldBackupsToZipFile: true
    backupBuildResults: false            # Don't backup build logs (large,
optional)
    backupBuildArchive: false            # Don't backup archived artifacts
(very large)
    backupUserContents: true             # Backup user-home plugin data

Limitations:
  - Does not backup while Jenkins is running (risk of partial/corrupt
backup)
  - Does not backup secrets/ directory reliably
  - Not suitable as sole backup mechanism — use alongside filesystem
backup
```

⚙️ Recovery Procedures

```
# ── RECOVERY SCENARIO 1: JCasC (fastest) ──────────────────────────
# Jenkins is completely gone. Start from scratch.

# 1. Start new Jenkins pod (same version as last known good)
kubectl apply -f jenkins-deployment.yaml

# 2. Restore secrets/ from secure store
kubectl exec jenkins-pod -- mkdir -p /var/jenkins_home/secrets
aws secretsmanager get-secret-value \
    --secret-id jenkins/master-key \
    --query SecretString --output text | \
    kubectl exec -i jenkins-pod -- tee
/var/jenkins_home/secrets/master.key

# 3. Mount casc.yaml (JCasC)
# Already mounted via ConfigMap from jenkins-config repo
```

```
# Jenkins reads it on startup → full system config restored

# 4. Wait for Jenkins to start (60-90 seconds)
kubectl wait --for=condition=ready pod/jenkins-0 --timeout=5m

# 5. Organization Folders auto-scan GitHub org
# All pipelines recreated in ~5 minutes

echo "Recovery complete. Total time: ~10 minutes"

# ── RECOVERY SCENARIO 2: Full filesystem restore ──────────────────
# 1. Stop Jenkins
kubectl scale deployment jenkins --replicas=0 -n jenkins

# 2. Download backup
TIMESTAMP="20240115_020000"
aws s3 cp s3://my-jenkins-backups/${TIMESTAMP}/jenkins-backup.tar.gz /tmp/

# 3. Restore to PVC
kubectl run restore-pod --image=alpine --restart=Never \
    --overrides='{"spec":{"volumes":
[{"name":"jenkins","persistentVolumeClaim":{"claimName":"jenkins-home-
pvc"}}],"containers":[{"name":"restore","image":"alpine","volumeMounts":
[{"name":"jenkins","mountPath":"/var/jenkins_home"}],"command":["sh","-
c","sleep 3600"]}]}}' -n jenkins

kubectl cp /tmp/jenkins-backup.tar.gz jenkins/restore-pod:/tmp/
kubectl exec restore-pod -n jenkins -- sh -c "
    rm -rf /var/jenkins_home/* &&
    tar xzf /tmp/jenkins-backup.tar.gz -C / &&
    echo 'Restore complete'
"
kubectl delete pod restore-pod -n jenkins

# 4. Start Jenkins
kubectl scale deployment jenkins --replicas=1 -n jenkins

echo "Recovery complete. Total time: ~30 minutes"
```

⚙️ Recovery Time Objectives

```
Define your RTO (Recovery Time Objective) and RPO (Recovery Point
Objective):

RTO = how long until Jenkins is functional after failure
RPO = how much data/config can be lost (time window)

Strategy                RTO         RPO         Complexity
──────────────────────────────────────────────────────────────────
JCasC + Git only        5-15 min    No config   Low: just restart Jenkins
```

```
                                        loss (Git   with same JCasC yaml
                                        is the      (build history lost)
                                        source)

JCasC + daily backup    10-30 min   < 24h loss  Medium: restore secrets/
                                                then apply JCasC

Daily filesystem backup 30-60 min   < 24h loss  Medium: restore
JENKINS_HOME
                                                from S3 backup

PVC snapshots           15-30 min   < 1h loss   Low: restore from snapshot
(hourly snapshots)                              create new PVC from
snapshot

Active-Active Jenkins   < 5 min     No loss     High: CloudBees HA
(licensed)
(CloudBees CI HA)                               (not available in OSS
Jenkins)

RECOMMENDATION for most teams:
  JCasC (primary) + daily PVC snapshot (for secrets/build history)
  RTO: 10-20 minutes
  RPO: < 24 hours for config, < 1 hour for secrets
  Cost: minimal (S3 storage for snapshots)
```

🔑 Key Concepts

Concept	Meaning
`secrets/master.key`	AES master key that decrypts credentials.xml — protect like root password
RTO	Recovery Time Objective — how long until service is restored
RPO	Recovery Point Objective — max acceptable data loss window
JCasC recovery	Apply jenkins.yaml → Jenkins fully configured — fastest path
PVC snapshot	Kubernetes/CSI point-in-time snapshot of JENKINS_HOME volume
ThinBackup	Plugin for scheduled incremental backups within Jenkins
Build discarder	Policy to auto-delete old build history — keeps JENKINS_HOME manageable
Organization Folder rescan	After restore, triggers auto-discovery of all pipelines
Atomic backup	Backup credentials.xml and secrets/ together — useless without both

---

💬 Short Crisp Interview Answer

> *"Jenkins backup strategy has a modern path and a traditional path. Modern: JCasC with configuration in Git — system config, credentials definitions, agent templates, plugin configs all version-controlled. Recovery is: start new Jenkins container, restore the `secrets/` directory (master encryption key), apply the JCasC YAML, and Organization Folders auto-discover all pipelines within 10 minutes. Build history is the one thing you don't get back, but for most teams that's acceptable. Traditional: nightly tar of JENKINS_HOME to S3 (excluding workspace/, logs/, and build archives which are large and transient), or Kubernetes VolumeSnapshot for PVC-backed deployments. Critical: the `secrets/` directory must be backed up separately and securely — without the master key, credentials.xml is unreadable even if you have the encrypted file. Store it in AWS Secrets Manager or Vault, not in the same S3 bucket as the Jenkins backup. Recovery time with JCasC: 10-15 minutes. With filesystem restore: 30-60 minutes."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`credentials.xml` without `secrets/master.key` = unreadable.** These two files are cryptographically linked. Backing up only `credentials.xml` and losing `secrets/` means all credentials are permanently lost — you can't decrypt them without the master key. Always back them up together, atomically. Store `secrets/` in a separate more-protected location (Vault/AWS Secrets Manager) than the rest of JENKINS_HOME.
- **Restoring to a different Jenkins version can corrupt the configuration.** Jenkins is not guaranteed to be backward-compatible across major versions. Restoring a Jenkins 2.387 backup into a 2.440 instance may cause plugin incompatibilities, XML format mismatches, or broken configurations. Always restore to the same version first, then upgrade.
- **Backup while Jenkins is running risks partial state.** If a backup runs while Jenkins is actively writing pipeline state (program.dat for running pipelines), you may capture partial file writes — the backup appears complete but is corrupt. For highest fidelity: stop Jenkins → backup → start Jenkins. For minimal-downtime: accept small corruption risk on rarely-needed backup, rely on JCasC for critical config.
- **Build artifacts in the backup explode its size.** Including `jobs/*/builds/*/archive/` in backups can make a JENKINS_HOME backup 100x larger than the configuration-only backup. Exclude artifacts from the backup — they should be in Nexus/Artifactory with proper retention. Only backup `jobs/*/config.xml` (job definitions) and `jobs/*/builds/*/build.xml` (build metadata) if you want build history.

Topic 8.5 — Monitoring Jenkins

🔴 Advanced | Observability for the CI Platform

📌 What It Is — In Simple Terms

Monitoring Jenkins means collecting, storing, and visualizing metrics that describe Jenkins' health, performance, and capacity — so that operators can detect problems before they impact engineering teams, understand trends in build times and queue depths, and make data-driven decisions about capacity scaling.

A Jenkins instance that is not monitored is one that surprises you with outages instead of giving you time to prevent them.

⚙️ Prometheus Metrics Plugin

```
Plugin ID: prometheus
What it does:
  - Exposes a Prometheus metrics endpoint at /prometheus
  - Jenkins Controller scrapes its own state and exposes as metrics
  - No external agent required — metrics come from Jenkins itself

Metrics endpoint:
  https://jenkins.example.com/prometheus/

Metrics are automatically scraped by Prometheus at a configured interval
(15-30s)
```

```yaml
# prometheus.yaml scrape config
scrape_configs:
  - job_name: 'jenkins'
    metrics_path: '/prometheus'
    scheme: 'https'
    tls_config:
      insecure_skip_verify: false
    static_configs:
      - targets: ['jenkins.example.com']
    # Optional: use Bearer token for auth
    bearer_token_file: /var/run/secrets/jenkins-metrics-token

  # Also monitor Jenkins agents if using node_exporter on persistent
agents:
  - job_name: 'jenkins-agents'
    static_configs:
      - targets:
        - 'agent-1.example.com:9100'
        - 'agent-2.example.com:9100'
```

⚙️ Key Jenkins Metrics — Complete Reference

```
HEALTH METRICS:
────────────────────────────────────────────────────────────────────────
jenkins_health_check_score
  Type: Gauge (0.0 to 1.0)
  Meaning: Overall health score (1.0 = all health checks passing)
  Alert: < 0.8 → at least one health check failing
```

```
default_jenkins_builds_health_score
  Type: Gauge
  Meaning: % of recent builds that succeeded

AVAILABILITY:
jenkins_up
  Type: Gauge (0 or 1)
  Meaning: Is Jenkins responding? (0 = down)
  Alert: == 0 for > 60s → page on-call

QUEUE METRICS (capacity planning):
────────────────────────────────────────────────────────────────────────────
jenkins_queue_size_value
  Type: Gauge (count of builds in queue)
  Meaning: How many builds are waiting for an agent
  Normal: 0-5 (briefly)
  Alert: > 20 for > 5 minutes → agent capacity insufficient

jenkins_queue_blocked_value
  Type: Gauge
  Meaning: Builds blocked (waiting for a specific resource/lock)
  Alert: > 5 for > 10 minutes → possible deadlock or resource starvation

jenkins_queue_buildable_value
  Type: Gauge
  Meaning: Builds ready to run but waiting for free agent executor
  Alert: Sustained > 10 → add more agents

jenkins_queue_waiting_value
  Type: Gauge
  Meaning: Builds in quiet period (not yet buildable)

EXECUTOR METRICS:
────────────────────────────────────────────────────────────────────────────
jenkins_executor_count_value
  Type: Gauge
  Meaning: Total executor count across all connected agents
  Use: Capacity baseline

jenkins_executor_in_use_value
  Type: Gauge
  Meaning: Currently busy executors
  Derived: utilization = in_use / count (want < 85% sustained)

jenkins_executor_free_value
  Type: Gauge
  Meaning: Available executors right now

BUILD METRICS:
────────────────────────────────────────────────────────────────────────────
jenkins_builds_success_build_count_total
  Type: Counter
  Rate: rate(jenkins_builds_success_build_count_total[5m])
  Meaning: Successful builds per second
```

```
jenkins_builds_failed_build_count_total
  Type: Counter
  Rate: rate(jenkins_builds_failed_build_count_total[5m])
  Alert: rate sustained > baseline → something broke

jenkins_builds_duration_milliseconds_summary{quantile="0.5"}
  Type: Summary (histogram with quantiles)
  Meaning: Median build duration
  Quantiles available: 0.5, 0.75, 0.95, 0.99
  Alert: p95 > 2x baseline → builds getting significantly slower

jenkins_builds_failed_build_count_total / jenkins_builds_total_count
(derived)
  Meaning: Build failure rate
  Alert: > 20% failure rate sustained → systemic problem

PLUGIN / AGENT METRICS:
────────────────────────────────────────────────────────────────────────
jenkins_node_count_value
  Type: Gauge
  Meaning: Total configured nodes/agents

jenkins_node_online_value
  Type: Gauge
  Meaning: Currently online (connected) agents
  Alert: < expected_agents → agents disconnecting

jenkins_plugins_active
  Type: Gauge
  Meaning: Active (enabled, loaded) plugin count
  Alert: Sudden change → plugin install/uninstall/failure

JVM METRICS (from jvm_* prefix - built into Prometheus plugin):
────────────────────────────────────────────────────────────────────────
jvm_memory_bytes_used{area="heap"}
  Alert: > 85% of max heap → risk of OOM

jvm_gc_collection_seconds_sum{gc="G1 Young Generation"}
  Meaning: Time spent in GC
  Alert: > 10% of total time in GC → heap too small

jvm_threads_current
  Meaning: Current thread count
  Alert: rapid growth → thread leak

process_cpu_seconds_total
  Rate: rate(process_cpu_seconds_total[5m])
  Alert: > 90% CPU sustained → Controller overloaded
```

⚙️ Grafana Dashboard — Complete Reference

```
Dashboard layout: Jenkins Operations

Row 1: Health Overview
  ┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
┌──────────────────┐
  │ Jenkins Status   │ │ Build Queue Depth│ │ Executor         │ │ Build
Success    │
  │ ● UP / ● DOWN    │ │ [gauge: 0-50]    │ │ Utilization      │ │ Rate
(24h)       │
  │ Uptime: 14d 3h   │ │ Current: 3       │ │ [gauge: 0-100%]  │ │ [stat:
94.2%]    │
  └──────────────────┘ └──────────────────┘ └──────────────────┘
└──────────────────┘

Row 2: Build Metrics (time series)
  ┌──────────────────────────────────────┐
┌──────────────────────────────────────┐
  │ Build Rate (builds/minute)           │ │ Build Duration (p50, p95)
│
  │ ─── successful  ─── failed           │ │ p50: 8.2min  p95: 23.4min
│
  │                     ↑spike here      │ │         /\
│
  │  ────────────────/─────────────────  │ │ ────────
────────────────────────  │
  └──────────────────────────────────────┘
└──────────────────────────────────────┘

Row 3: Queue Depth (capacity planning)
  ┌──────────────────────────────────────┐
┌──────────────────────────────────────┐
  │ Queue Depth Over Time                │ │ Queue Wait Time (p95)
│
  │ Alert threshold: 20 ──────────────── │ │
│
  │         /\                           │ │
──────────────────────────────────────  │
  │ ────────  ──────────────────────────│ │ Normal: < 60s | Current: 23s
│
  └──────────────────────────────────────┘
└──────────────────────────────────────┘

Row 4: JVM Health
  ┌──────────────────────────────────────┐
┌──────────────────────────────────────┐
  │ Heap Usage (%)                       │ │ GC Pause Time
│
  │ Max: 16GB | Alert: 85%               │ │ G1 Young: avg 45ms
│
  │ Current: ═══════════╬═══ 78%         │ │ G1 Old:   avg 210ms
│
  │                                      │ │ Alert: > 500ms
```

```
│
  └──────────────────────────────────────────┘
└──────────────────────────────────────────┘
```

```
// Example Grafana panel: Build Queue Depth with alert threshold
{
  "title": "Build Queue Depth",
  "type": "timeseries",
  "targets": [
    {
      "expr": "jenkins_queue_size_value",
      "legendFormat": "Total Queue"
    },
    {
      "expr": "jenkins_queue_buildable_value",
      "legendFormat": "Buildable (waiting for executor)"
    },
    {
      "expr": "jenkins_queue_blocked_value",
      "legendFormat": "Blocked"
    }
  ],
  "thresholds": {
    "mode": "absolute",
    "steps": [
      {"value": null,  "color": "green"},
      {"value": 10,    "color": "yellow"},
      {"value": 20,    "color": "red"}
    ]
  },
  "alert": {
    "conditions": [
      {
        "evaluator": {"type": "gt", "params": [20]},
        "operator":  {"type": "and"},
        "query":     {"model": {"expr": "jenkins_queue_size_value"}},
        "reducer":   {"type": "avg"},
        "type":      "query"
      }
    ],
    "for": "5m",
    "name": "Jenkins Queue Depth Critical",
    "message": "Jenkins build queue has exceeded 20 builds for 5+ minutes
— agent capacity insufficient"
  }
}
```

⚙️ Alerting Rules (Prometheus AlertManager)

```yaml
# jenkins-alerts.yaml — AlertManager rules

groups:
  - name: jenkins.critical
    interval: 30s
    rules:
      # Jenkins is DOWN
      - alert: JenkinsDown
        expr: jenkins_up == 0
        for: 1m
        labels:
          severity: critical
          team: platform
        annotations:
          summary: "Jenkins is DOWN"
          description: "Jenkins Controller has been unreachable for 1+
minutes"
          runbook: "https://wiki.example.com/runbooks/jenkins-down"

      # Heap critical
      - alert: JenkinsHeapCritical
        expr: |
          jvm_memory_bytes_used{area="heap"} /
          jvm_memory_bytes_max{area="heap"} > 0.90
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Jenkins JVM heap usage critical (> 90%)"
          description: "Heap at {{ humanizePercentage $value }} — OOM
risk"
          action: "Restart Jenkins or increase heap size"

  - name: jenkins.warning
    interval: 60s
    rules:
      # Queue too deep
      - alert: JenkinsBuildQueueDepth
        expr: jenkins_queue_size_value > 20
        for: 5m
        labels:
          severity: warning
          team: platform
        annotations:
          summary: "Jenkins build queue depth critical ({{ $value }}
builds waiting)"
          description: "Build queue has been > 20 for 5+ minutes — agents
cannot keep up"
          action: "Check Kubernetes cluster autoscaler. Scale up agent
pool if needed."

      # Build failure rate spike
      - alert: JenkinsBuildFailureRateHigh
```

```
        expr: |
          rate(jenkins_builds_failed_build_count_total[15m]) /
          rate(jenkins_builds_total_count[15m]) > 0.3
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Jenkins build failure rate high ({{ humanizePercentage
$value }})"
          description: "More than 30% of builds failing in the last 15
minutes"

      # Heap warning
      - alert: JenkinsHeapHigh
        expr: |
          jvm_memory_bytes_used{area="heap"} /
          jvm_memory_bytes_max{area="heap"} > 0.80
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "Jenkins JVM heap usage high (> 80%)"

      # GC pauses
      - alert: JenkinsGCPauseHigh
        expr: |
          rate(jvm_gc_collection_seconds_sum[5m]) /
          rate(jvm_gc_collection_seconds_count[5m]) > 0.5
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Jenkins GC pause time high (> 500ms avg)"
          description: "Average GC pause {{ $value | humanizeDuration }} —
may cause UI lag"

      # Agents offline
      - alert: JenkinsAgentsDown
        expr: jenkins_node_online_value < jenkins_node_count_value * 0.8
        for: 3m
        labels:
          severity: warning
        annotations:
          summary: "Jenkins: >20% of agents are offline"
          description: "{{ $value }} agents online out of expected"
```

⚙️ Custom Build Metrics — DORA Metrics

```
// Push custom build metrics to Prometheus via Pushgateway
// Enables DORA metric tracking:
```

```
// - Deployment Frequency: how often you deploy to production
// - Lead Time: time from commit to production deploy
// - Change Failure Rate: % of deployments causing failure
// - Mean Time to Recovery: how long to fix a failed deployment

stage('Record Deploy Metrics') {
    steps {
        script {
            def deploymentTime = System.currentTimeMillis()
            def commitTime     = sh(
                script: 'git log -1 --format=%ct HEAD',
                returnStdout: true
            ).trim().toLong() * 1000  // Convert to milliseconds

            def leadTimeSeconds = (deploymentTime - commitTime) / 1000

            withCredentials([string(credentialsId: 'pushgateway-url',
variable: 'PG_URL')]) {
                sh """
                    # Push deployment frequency metric
                    cat <<EOF | curl -s --data-binary @-
\$PG_URL/metrics/job/jenkins/instance/${env.JOB_NAME}
jenkins_deployment_total{env="production",service="${env.APP_NAME}"} 1
jenkins_lead_time_seconds{env="production",service="${env.APP_NAME}"}
${leadTimeSeconds}
EOF
                """
            }
        }
    }
}
```

⚙️ Disk Usage Monitoring

```
// Monitor JENKINS_HOME disk usage — a common silent failure mode
// Disk full → builds fail cryptically, logs truncated, state corruption

pipeline {
    triggers { cron('H * * * *') }  // Check every hour

    stages {
        stage('Check Disk Usage') {
            steps {
                script {
                    def diskUsage = sh(
                        script: 'df /var/jenkins_home --output=pcent |
tail -1 | tr -d " %"',
                        returnStdout: true
                    ).trim().toInteger()
```

```
                    echo "JENKINS_HOME disk usage: ${diskUsage}%"

                    if (diskUsage > 90) {
                        // Critical: stop taking new builds?
                        // Actually Jenkins auto-detects low disk and
stops builds
                        slackSend channel: '#ops-critical', color:
'danger',
                                  message: "🔴 Jenkins disk usage
CRITICAL: ${diskUsage}%"
                        error("Disk usage critical: ${diskUsage}%")
                    } else if (diskUsage > 75) {
                        slackSend channel: '#ops-alerts', color:
'warning',
                                  message: "⚠ Jenkins disk usage high:
${diskUsage}%"
                        unstable("Disk usage warning: ${diskUsage}%")
                    }
                }
            }
        }

        stage('Cleanup Old Builds') {
            when {
                expression {
                    def usage = sh(script: 'df /var/jenkins_home --
output=pcent | tail -1 | tr -d " %"',
                                   returnStdout: true).trim().toInteger()
                    return usage > 70
                }
            }
            steps {
                // Force cleanup of old builds beyond retention policy
                sh """
                    find /var/jenkins_home/jobs -name "builds" -type d |
while read dir; do
                        # Keep last 10 builds per job
                        ls -dt "\$dir"/[0-9]* | tail -n +11 | xargs rm -rf
2>/dev/null || true
                    done
                """
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning

Concept	Meaning
Prometheus plugin	Exposes Jenkins metrics at `/prometheus` — scrape with Prometheus
`jenkins_queue_size_value`	Current build queue depth — key capacity indicator
`jenkins_executor_in_use_value`	Currently busy executors — utilization signal
`jvm_memory_bytes_used{area="heap"}`	Current heap usage — OOM risk indicator
`jenkins_builds_duration_milliseconds_summary`	Build duration histogram with quantiles
Pushgateway	Prometheus component for receiving pushed metrics from batch jobs
AlertManager	Prometheus component that routes alerts to Slack/PagerDuty/email
DORA metrics	Deployment Frequency, Lead Time, Change Failure Rate, MTTR
p95 build time	95th percentile — slow tail of builds that engineers notice
GC overhead	% of JVM time in GC — > 10% indicates heap pressure
`jenkins_up`	Binary metric: 1=Jenkins responding, 0=Jenkins down

💬 Short Crisp Interview Answer

> *"Jenkins monitoring uses the Prometheus plugin, which exposes a `/prometheus` endpoint that Prometheus scrapes every 15-30 seconds. The key metrics to watch: queue depth (`jenkins_queue_size_value` — sustained > 20 means agents can't keep up), executor utilization (in-use divided by total — above 85% means you need more agents), JVM heap usage (`jvm_memory_bytes_used{area='heap'}` — alert at 80%, page at 90%), and build failure rate. GC monitoring is critical: if `jvm_gc_collection_seconds_sum` shows the JVM spending > 10% of time in GC, the heap is undersized and you'll see UI lag. The Grafana dashboard has four rows: health overview (queue, utilization, success rate), build metrics time series (build rate and duration trends with p50/p95), capacity planning (queue depth with alert thresholds), and JVM health (heap usage, GC pauses). Alerting: JenkinsDown fires after 1 minute, heap > 90% is critical/page, queue > 20 for 5 minutes is a warning. For DORA metrics, push custom metrics via Prometheus Pushgateway — lead time from commit-to-deploy is the most valuable one for measuring CI/CD effectiveness."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ `jenkins_executor_count_value` **includes offline agents.** If an agent disconnects, its executors are still counted in `executor_count_value` but not in `executor_in_use_value`. Utilization calculated as `in_use / count` appears low even though available capacity has actually dropped. Better metric: `jenkins_executor_in_use_value / jenkins_executor_free_value + jenkins_executor_in_use_value`.
- **Prometheus plugin scrape timeout.** On a loaded Jenkins with thousands of jobs and many running builds, generating the metrics response can take 10-30 seconds. If Prometheus's `scrape_timeout` is shorter (default 10s), scrapes silently fail and you lose metric continuity. Increase Prometheus's `scrape_timeout` for Jenkins: `scrape_timeout: 30s`.
- **Build duration metrics don't tell you WHERE time is spent.** `jenkins_builds_duration_milliseconds_summary` tells you the total build duration. It doesn't tell you which stage is slow. Add stage-level timing to your Shared Library and push to Prometheus via Pushgateway — this is what separates basic monitoring from actionable observability.
- **Disk usage alert before the crisis.** Jenkins silently handles low-disk by refusing to start new builds (to prevent corruption). But by the time it does this, disk is already above 95%. Alert at 75% to give yourself a response window. The builds discarded by the Build Discarder run asynchronously — they may not free space fast enough once you're in emergency territory.

📊 Category 8 Summary — Quick Reference

Topic	Core Concept	Interview Priority
8.1 JVM Performance Tuning	G1GC + right-sized heap; UseContainerSupport; numExecutors=0; GC logging	⭐⭐⭐⭐⭐
8.2 Build Time Optimization	PVC cache; parallel stages; shallow+sparse clone; layer-ordered Dockerfile	⭐⭐⭐⭐⭐
8.3 Ephemeral vs Persistent ⚠	Ephemeral=clean/elastic; Persistent=hardware/cache; hybrid for production	⭐⭐⭐⭐
8.4 Backup and Recovery	JCasC=fastest; secrets/master.key=most critical; PVC snapshots; RTO/RPO	⭐⭐⭐⭐
8.5 Monitoring Jenkins	Prometheus plugin; queue/heap/GC metrics; Grafana dashboard; DORA	⭐⭐⭐⭐⭐

🔑 The Mental Model for Category 8

```
PERFORMANCE follows a hierarchy:
  1. Right-size JVM heap (too small → OOM; too large → wasteful)
  2. Choose right GC (G1GC for most; ZGC for huge heaps)
  3. Zero executors on Controller (protect it for coordination)
  4. Eliminate build bottlenecks (caching > parallelism > incrementality)
  5. Monitor to know where the bottleneck actually is
```

```
AGENT ARCHITECTURE decision matrix:
  Standard CI builds → Ephemeral (K8s pods): always
  iOS / macOS builds → Persistent (Mac hardware): required
  GPU / ML builds    → Persistent or cloud GPU: required
  Default position: Ephemeral. Add Persistent only for justified
exceptions.

BACKUP defense-in-depth:
  Layer 1: JCasC in Git (config is never lost — it's code)
  Layer 2: secrets/ in Vault/Secrets Manager (separately from config)
  Layer 3: Daily PVC snapshot (for build history if needed)
  Layer 4: Build discarder (keep JENKINS_HOME manageable)
  Recovery test: restore to dev Jenkins quarterly — backup you haven't
tested is not a backup

MONITORING the four horsemen of Jenkins trouble:
  Queue depth growing → capacity problem → scale agents
  Heap usage rising   → memory pressure → increase -Xmx or fix leak
  GC time spiking     → heap too small → increase heap or tune GC
  Build time rising   → bottleneck somewhere → check stage-level timing
```

🧪 Self-Quiz — 10 Interview Questions

1. Your Jenkins Controller is experiencing intermittent 30-60 second UI freezes during peak hours. What is the most likely cause and how do you diagnose it?

2. Why should `numExecutors` always be 0 on the Jenkins Controller? What happens if you don't do this?

3. A 45-minute Maven build needs to be cut to under 10 minutes. Walk through your optimization strategy in priority order.

4. Compare ephemeral and persistent agents on security, cost, and reproducibility. When would you use persistent agents in 2024?

5. What is `secrets/master.key` and why must it be backed up separately from `credentials.xml`?

6. Jenkins is running in Kubernetes. A developer says "I don't need `-Xmx` because Kubernetes limits memory." Why is this wrong and what's the correct configuration?

7. The Jenkins build queue depth has been consistently > 30 builds for the past hour. Walk through your incident response.

8. What metrics would you put on a Jenkins operational dashboard? Name 8 specific metrics and what thresholds you'd alert on.

9. Explain the JCasC recovery procedure. From "Jenkins is completely gone" to "fully operational." What are the steps and estimated time?

10. Your monorepo has 50 services. Every commit to any file triggers a full build of all 50 services (total: 3 hours). How do you fix this?

Next: Category 10 — Modern CI/CD Ecosystem Awareness Or: "quiz me" to test yourself on Category 8

> **Document:** Category 8 Complete | Scaling, Performance & Reliability **Coverage:** 5 topics | Intermediate → Advanced **Key insight:** Performance is not a feature you add at the end — it's the discipline of measuring first, then fixing the actual bottleneck, not the assumed one.